ENGINEERING CHANGE NOTE (ECN)

PROJECT: PROJECT-MYMILLI (revision of PROJECT-CE)

ORIGINATION: Prof. Kidd, Instructor (ECO 0.3, 3/1/2030)

PURPOSE OF CHANGE: Replace millis() with mymilli(), driven by an ISR and register constants written by the team

DOCUMENT NUMBER: ECN0.3

DATE OF PUBLICATION: October 15, 2026

LEAD ENGINEER: Raheem Provost, UMBC

TEAM LEAD: Raheem Provost

EXECUTIVE SUMMARY: This ECN implements ECO 0.3 by replacing the millisecond clock supplied by arduino.h with a function written by the team. The LED blink tasks of PROJECT-CE now read time from `mymilli()` instead of `millis()`. The count behind `mymilli()` is kept by a compare-match interrupt service routine (ISR) on Timer/Counter0, run in CTC mode with a compare value of 249 and a divide-by-64 clock, so it fires exactly once per millisecond. The time base lives in two new files, `mymilli.h` and `mymilli.cpp`, and every timer register address and bit position it uses is defined by a constant in `mymilli.cpp`. Because `mymilliInit()` reprograms Timer/Counter0, the Arduino `millis()`, `micros()` and `delay()` no longer work in this sketch. The customer requirements, high-level technical requirements, hardware and user-visible behavior are unchanged. The System Design Document is revised to Rev 3. Three educational constraints (EC.7, EC.7.1, EC.7.2) and three tests (T.8 to T.10) are added, and the #include check of T.6 and T.7 is changed.

HIGH-LEVEL DESCRIPTION: This ECN notifies stakeholders that PROJECT-MYMILLI takes its time from `mymilli()`, a function and time base owned by the team, instead of the `millis()` function from the third-party arduino.h used in PROJECT-CE and PROJECT-ASYNC.

REASON FOR CHANGE: ECO 0.3 requires a design that is more flexible, so the team can customize and extend the time function; entirely under company control, isolated from any future change in the third-party arduino.h; and free of possible future intellectual property ownership claims. ECO 0.3 accepts some added initial cost for independence, extensibility and reuse that should reduce labor and cost in future projects.

CUSTOMER REQUIREMENTS:

The customer requirements, C1 through C6, are identical to those of PROJECT-ASYNC. See [1] REQUIREMENTS section, PART I, Table 1. ECO 0.3 makes no change to them.

HIGH-LEVEL TECHNICAL REQUIREMENTS:

The high-level technical requirements, HL.1 through HL.6, are identical to those of PROJECT-ASYNC. See [1] REQUIREMENTS section, PART II, Table 3. The educational constraints, EC.1 through EC.5, are also identical; see [1] REQUIREMENTS section, PART II, Table 4. EC.6, added by ECN 0.1, is unchanged; see [2] REQUIREMENTS section, Table 2. Three educational constraints, EC.7 to EC.7.2, are added by this ECN.

Changed Documents: provost.cmpe311.fall26.project#3 (System Design Document, Rev 3, revising provost.cmpe311.fall26.project#2 [2]). Deleted text is ~~struck out~~ and additions are **marked**, each with the date.

- **10/15/26 (REQUIREMENTS, Table 2)** EC.7 added: "The Developer must replace the millis() function provided by arduino.h with a function written by The Developer and named mymilli(). The replacement may extend the capability of millis()." (ECO 0.3, Educational Constraint 1.0)
- **10/15/26 (REQUIREMENTS, Table 2)** EC.7.1 added: "The Developer must write their own PWM counter ISR and use it as the timing mechanism underlying mymilli()." (ECO 0.3, Educational Constraint 1.0.1)
- **10/15/26 (REQUIREMENTS, Table 2)** EC.7.2 added: "The Developer must define their own register constants that would normally be defined in avr.h. The constants need not have the same names as those in avr.h." (ECO 0.3, Educational Constraint 1.0.2)
- **10/15/26 (REQUIREMENTS, Table 3)** Table 3 added: it maps each item of ECO 0.3 (1.0 to 1.3) to where the revised document meets it.
- **10/15/26 (TESTING AND VALIDATION REQUIREMENTS, Table 6)** T.8 added: "The sketch files (the .ino file, mymilli.h and mymilli.cpp) must contain no call to millis(), micros() or delay(). mymilli() must be declared in mymilli.h and defined in mymilli.cpp, and the blink functions must use it as their only time source." Verifies EC.7. T.9 added: own Timer/Counter0 compare-match ISR in CTC mode that advances the count once per millisecond, builds with no duplicate-vector error, and the count advances. Verifies EC.7.1, EC.7. T.10 added: every timer register address and timer bit position defined by a constant in mymilli.cpp, no AVR-defined timer name present. Verifies EC.7.2. T.1 through T.7 are re-run on the PROJECT-MYMILLI build.
- **10/15/26 (TEST PROCEDURES, T.6 and T.7)** The search for #include expected no matches. It now finds exactly one in the sketch, `#include "mymilli.h"`, a header written by The Developer. Pass criterion: ~~no #include lines~~ **the only #include in the .ino file is "mymilli.h" and no library or scheduler header is included**.
- **10/15/26 (REQUIREMENTS TRACEABILITY, Table 7)** EC.7 row added: verified by T.8. EC.7.1 row added: verified by T.9. EC.7.2 row added: verified by T.10.
- **10/15/26 (SYSTEM OVERVIEW)** Added: the two LED tasks now read time from `mymilli()`, whose count is kept by an ISR written by The Developer, instead of from `millis()`, kept by the Arduino core. The task split and dispatch are unchanged.
- **10/15/26 (SOFTWARE DESIGN)** Each LED keeps its own last-toggle time and toggles when ~~millis()~~ **mymilli()** reaches that time plus its interval. Added: the time base in two new files, `mymilli.h` and `mymilli.cpp`; `#include "mymilli.h"` at the top of the sketch and a `mymilliInit()` call at the end of `setup()`; timer selection (Timer/Counter0, whose compare-match A vector is free); timer configuration (CTC mode, compare value 249, clock / 64, tick of exactly 1,000 µs); the compare-match ISR, which adds 1 to `millicount`; Table 4 (time base parameters); Table 5 (register constants and the hardware items they stand for); start-up through `mymilliInit()`; atomic reading of the 32-bit count; the Arduino timing functions that no longer work; Figure 1 (time base data flow).
- **10/15/26 (HARDWARE DESIGN)** No change. Added note: Timer/Counter0 is internal; none of its output pins (5 and 6) is connected to the System.
- **10/15/26 (DESIGN RATIONALE AND ASSUMPTIONS)** Run time. The ~~millis()~~ **mymilli()** comparison is not rollover-safe, so continuous operation is supported for up to about 49.7 days. Added: why Timer/Counter0 in CTC mode, same resolution as the original with no skipped values, accuracy and jitter, estimated processor load, resources given up (millis(), micros(), delay() and analogWrite() on pins 5 and 6), and the single #include "mymilli.h". The other bullets are unchanged.

- **10/15/26 (APPENDIX B, DESIGN CODE)** PROJECT-CE source replaced by the PROJECT-MYMILLI source in three files: the sketch, `mymilli.h` and `mymilli.cpp`. The sketch adds `#include "mymilli.h"`, calls `mymilliInit()` at the end of `setup()`, and `blinkLED1()` and `blinkLED2()` call `mymilli()` instead of `millis()`. `mymilli.cpp` adds the register constants, the compare-match ISR, `mymilliInit()` and `mymilli()`.
- **10/15/26 (TITLE AND DATE, REFERENCES AND GLOSSARY, TESTING RESULTS, APPENDIX A)** Document reference changed to provost.cmpe311.fall26.project#3; revision history, references and new terms added; results and evidence to be re-recorded for the PROJECT-MYMILLI build, with rows added for T.8 to T.10.

Sections not listed above (customer requirements, HL.1 through HL.6, EC.1 through EC.6, bill of materials, hardware wiring, pin assignments, task table, dispatcher, serial dialog, T.1 through T.5) are unchanged and are cited to [1] and [2] in the revised document.

Added Documents: ECN0.3, provost.cmpe311.fall26.project#3.ECN (this note), added 10/15/26. mymilli.h and mymilli.cpp, and the updated PROJECT-MYMILLI sketch, added to the GitHub repository https://github.com/raheemprovost/CMPE311Project3 on 10/15/26.

Removed Documents: None.

Replaced Documents: None. provost.cmpe311.fall26.project#2 [2] remains the PROJECT-CE baseline and provost.cmpe311.fall26.project#1 [1] remains the PROJECT-ASYNC baseline.

Any Additional Design Considerations: No hardware change: the board, LEDs, 330 Ω resistors and pins 2 and 3 are as in [1]. Timer/Counter0 is now reserved for `mymilli()`: teams reusing it must add `mymilli.h` and `mymilli.cpp` to the sketch folder, include `mymilli.h`, and call `mymilliInit()` once in `setup()`. Because `mymilliInit()` reprograms Timer/Counter0, the Arduino `millis()`, `micros()` and `delay()` stop working, and so do libraries that depend on them, and `analogWrite()` on pins 5 and 6 cannot be used. The tick is exactly 1 ms (16 MHz ÷ 64 ÷ 250), so the count has 1 ms resolution and advances by 1 at a time. Entered intervals must be 1 to 32,767 ms because they are parsed into a 16-bit `int`, and continuous operation is supported for up to about 49.7 days because the `mymilli()` comparison is not rollover-safe. The time base assumes a 16 MHz clock.

REFERENCES:

[1] provost.cmpe311.fall26.project#1, CMPE311 Project 1 design document (PROJECT-ASYNC), Revision 1, September 22, 2026.

[2] provost.cmpe311.fall26.project#2, CMPE311 Project 2 design document (PROJECT-CE), Revision 2, October 3, 2026.

[3] provost.cmpe311.fall26.project#3, CMPE311 Project 3 design document (PROJECT-MYMILLI), Revision 3, October 15, 2026.

[4] PROJECT#3: PROJECT-MYMILLI assignment, including Engineering Change Order ECO 0.3 (3/1/2030, requestor Prof. Kidd), CMPE311.